

task_struct

מבנה הנתונים task_struct (המוגדר בקובץ include/linux/sched.h בליבת לינוקס) הוא אחד ממבני הנתונים הגדולים, המרכזיים והחשובים ביותר בקרנל. הוא מייצג את כל המטא-דאטה, ההקשר (Context), המשאבים והסטטוס של תהליך או חוט (Thread) בודד במערכת.

בגרסאות מודרניות כמו **Linux Kernel 6.14**, ה- task_struct מכיל מאות שדות ואלפי שורות קוד. להלן הצגה של השדות הארכיטקטוניים המרכזיים והקרדינליים ביותר בתוך מבנה זה, לצד ניתוח מעמיק לפי הדרישות שהגדרת.

ייצוג של מבנה ה- task_struct

```
struct task_struct {
    unsigned int __state;           // (Running, Sleeping, וכו') מצב התהליך הנוכחי
    pid_t pid;                      // (Process ID) מזהה התהליך
    pid_t tgid;                    // (Thread Group ID) מזהה קבוצת הנימים

    struct task_struct *real_parent; // מצביע לאב האמיתי של התהליך
    struct task_struct *parent;      // (ptrace למשל אחרי) מצביע לאב הנוכחי
    struct list_head children;       // רשימת הבנים של התהליך
    struct list_head sibling;        // רשימת האחים (תהליכים באותו אב)

    struct mm_struct *mm;           // (Virtual Memory) מצביע למרחב הזיכרון הווירטואלי
    struct mm_struct *active_mm;    // משמש תהליכי קרנל לניהול זיכרון זמני

    struct fs_struct *fs;           // (תיקיית שורש, CWD) הקשר מערכת קבצים
    struct files_struct *files;     // (FD Table) טבלת תיאורי הקבצים הפתוחים

    struct sched_entity se;         // (EEVDF/CFS) יישות תזמון עבור מתזמן הליבה
    void *stack;                   // (Kernel Stack) פוינטר למחסנית הליבה

    struct signal_struct *signal;   // משותף לקבוצת נימים (Signals) ניהול אותות
    struct sighand_struct *sighand; // מטפלי אותות
};
```

ניתוח מפורט של השדות המרכזיים

1. שדה המצב: state (או state)

• מה משמעות הערך הנשמר בו?

שדה זה מכיל מסיכת ביטים (Bitmask) או מספר שלם המציין את מצבו הריאלי הנוכחי של התהליך במערכת ההפעלה (למשל: TASK_RUNNING כאשר הוא רץ או ממתין בתור מעבד, TASK_UNINTERRUPTIBLE / TASK_INTERRUPTIBLE כאשר הוא ישן וממתין לאירוע או לדיסק, או EXIT_ZOMBIE / TASK_STOPPED).

2. מזהה התהליך: pid (Process ID)

• מה משמעות הערך הנשמר בו?

מספר שלם ייחודי המזהה את התהליך או הנימה (Thread) הספציפיים הללו מול הקרנל (למשל, PID 4521).

3. מזהה קבוצת הנימים: tgid (Thread Group ID)

• מה משמעות הערך הנשמר בו?

זהו המזהה המשותף לכל הנימים (Threads) השייכים לאותה תוכנית אב (Process). ברוב המקרים, ה-tgid של הנימה הראשית שווה ל-PID שלה.

4. עץ המשפחה: sibling, children, parent, real_parent

• מה משמעות הערך הנשמר בו?

מצביעים המקשרים את ה-task_struct הנוכחי אל האב שיצר אותו (parent), אל הבנים שנולדו ממנו (children), ואל תהליכים אחים חולקים את אותו אב (sibling).

5. מרחב הזיכרון הווירטואלי: mm ו-active_mm

• מה משמעות הערך הנשמר בו?

מצביע למבנה הנתונים struct mm_struct, המכיל את כל הגדרות מרחב הזיכרון הווירטואלי של התהליך (טבלאות הדפים, אזורי ה-Heap, ה-Stack, והקטעים הבינאריים).

6. הקשר מערכת הקבצים: fs

• מה משמעות הערך הנשמר בו?

מצביע למבנה הנתונים struct fs_struct, אשר שומר את המידע הגיאוגרפי של התהליך במערכת הקבצים: תיקיית העבודה הנוכחית (CWD - Present Working Directory) ותיקיית השורש של התהליך (root).

• מדוע שמירת ערך זה חשובה/הכרחי עבור הקרנל?

כאשר תהליך מריץ קריאת מערכת עם נתיב יחסי (Relative Path), הקרנל חייב לדעת היכן התהליך "נמצא" בעץ הספריות כדי להתחיל את אלגוריתם ה-Path Resolution מנקודת המוצא הנכונה (pwd<-fs<-current).

• מתי הערך מחושב ונשמר בשדה?

מועתק מהאב בעת יצירת התהליך ב-fork(), ומתעדכן בכל פעם שהתהליך מריץ קריאת מערכת מסוג chdir().

7. טבלת תיאורי הקבצים: files

- מה משמעות הערך הנשמר בו?

מצביע למבנה הנתונים struct files_struct, המנהל את מערך תיאורי הקבצים (FD Table) של התהליך – כלומר, מפת הביטים ומערך המצביעים לכל הקבצים הפתוחים (struct file *).

- מדוע שמירת ערך זה חשובה/הכרחי עבור הקרנל?

שדה זה מקשר בין מספרי ה-FD שהתוכנית מכירה במרחב המשתמש (למשל $fd = 3$) לבין אובייקטי הקובץ הפתוחים בזיכרון הליבה. בלעדיו, הקרנל לא יוכל לבצע פעולות read, write או close.

- מתי הערך מחושב ונשמר בשדה?

מוקצה בעת יצירת התהליך (ומשוכפל או משותף בהתאם לדגלי ה-fork/clone), ומתעדכן בכל פעם שהתהליך פותח קובץ חדש (open) או סוגר קובץ (close).

8. יישות התזמון: se (Scheduling Entity)

- מה משמעות הערך הנשמר בו?

מבנה מסוג struct sched_entity, המרכז את כל נתוני הזמן, העדיפות, והזמן הווירטואלי (Virtual Runtime) שהתהליך צרך במעבד.